Object-Oriented Programming in Java

Java Generics

Exercises

8 hands-on coding exercises on the Generics slide deck (J05), numbered as Part 6: generic classes and methods, bounds, wildcards, erasure, generic functional interfaces and comparators. You build your own generic list and sorted table, then use them in a complete mock exam. Difficulty rises from ★ to ★★★.

Contents

How to use this sheet **2**

Part 6 – Generics (J05) **2**
6.1 ★ Generic pairs 2
6.2 ★★ Generic methods on arrays 4
6.3 ★★ Your own generic list 5
6.4 ★★ Your own sorted registry 7
6.5 ★★★ Generic functional helpers 9
6.6 ★★★ Comparator factories, by hand and from the library 11
6.7 ★★★ Wildcards and erasure 13
6.8 ★★★ Mock exam: a library 15

How to use this sheet

8 coding exercises on the Generics slide deck (J05), numbered as Part 6 after the 28 exercises on J01–J04, and getting harder: ★ warm-up, ★★ core, ★★★ challenge. Solutions are in the companion PDF *Java Generics – Solutions*, and the theory each exercise needs is in *Java Generics – Theory notes*.

The exercises are written like the exam texts of Parts 2–5: each one describes a small system (a football league, a shop, a waiting list, a lab…) and asks for it in requirements R1, R2, …; the *Concepts* line under the title names the topics of generics it trains, and the *Hint* paragraphs point to the tools you need. As in Parts 2–5 there is one package per exercise (`ex6_1`, …), exact formats and messages, checked exceptions, and a `Main` whose body is the code before each sample run. Since the collections are not in the slides yet, 6.3 and 6.4 have you build your own generic list (`SimpleList`) and sorted table (`Registry`), with the method names of the library's `List` and, mostly, `Map` (`keys()` plays the role of `keySet()`); the later exercises reuse them, so import them from their packages. 6.8 is a complete mock exam, about two hours, with exam-style tests in the solutions.

Use Java 25. When the demo calls methods that throw a checked exception without catching it, the text says *main declared with: throws* ….

Exercise	Topics
6.1	generic classes, several type parameters, the diamond, generic methods, static factories, `equals` in a generic class
6.2	generic methods on arrays, bounded types, `Comparable<? super T>`, `Comparator<? super T>`, `Predicate`, no `new T[]`, array covariance
6.3	a generic container on `Object[]`, unchecked casts, `@SafeVarargs`, generic `Iterable`/`Iterator`, an API like `List`
6.4	bounded type parameters on a class, static nested generic classes, binary search, an API like `Map`, non-generic exceptions
6.5	`Function`, `UnaryOperator`, `BinaryOperator`, `Predicate`, `Consumer`, `BiConsumer`, `Supplier`, `ToDoubleFunction`, `and`/`or`/`negate`/`andThen`/`compose`, an anonymous class with state, grouping
6.6	`Comparator.comparing`, `comparingInt`, `reversed`, `thenComparing`, writing them by hand, overflow, top-n
6.7	invariance, `? extends`, `? super`, `?`, PECS, several bounds (`&`), type erasure, array generators
6.8	a mock exam: everything above, plus an enum and exam-style tests

Part 6 – Generics (J05)

6.1 ★ Generic pairs

Concepts: generic classes, type parameters and type arguments, the diamond, several type parameters, generic methods and static factories, `equals` and `hashCode` in a generic class.

A football league needs small objects that hold two values together: a match is a pair of teams, a result is a pair of goal counts, a line of the league table is a team with its points. Writing a class for each of them would repeat the same code with different types; the slides' answer is a generic class, written once and used with any type. All the classes are in the package `ex6_1`, and the class `Main` contains the demo below.

R1 – Pairs of the same type

The generic class `Pair<T>` holds two values of the same type `T`: a match is a `Pair<String>` (home team, away team), a result a `Pair<Integer>` (home goals, away goals). The type argument must be a

class, so the goals are `Integer`s, not `int`s; with the diamond, `new Pair<>(2, 1)`, the compiler infers it. Start from the slides' version, with the constructor `Pair(T a, T b)`, the getters `T first()` and `T second()`, and the setters `void set1st(T x)` and `void set2nd(T x)`, which the league uses to correct a result after the referee's report.

Add the static factory method `static <T> Pair<T> of(T a, T b)`, so that `Pair.of("Juventus", "Torino")` builds a pair without writing the type argument. The method `boolean contains(T x)` tells whether one of the two values equals `x` (a team plays in a match), and must work when the values are `null`. The method `Pair<T> swapped()` returns a new pair with the two values exchanged: the return match, played at the other team's ground. `toString()` returns the two values in round brackets, e.g. `"(Juventus, Torino)"`.

Hint: `Objects.equals(a, b)` compares two references that may be `null`.

R2 – Pairs of different types

A line of the league table holds a team and its points, which have different types. The generic class `Couple<A, B>` has two type parameters, a constructor `Couple(A first, B second)`, the getters `A first()` and `B second()`, and its values never change. The method `Couple<B, A> swap()` returns a new couple with the values, and therefore the types, exchanged: `<42, Juventus>` is a `Couple<Integer, String>`. The generic method `<C> Couple<A, C> withSecond(C value)` returns a new couple with the same first value and a second value of any type, for example the team with its average goals instead of its points. Two couples are equal when both their values are equal, so `Couple` redefines `equals` and `hashCode`. `toString()` uses angle brackets: `"<Juventus, 42>"`.

R3 – The league table

A method returns a single value; a pair lets it return two, with no casts for the caller. The utility class `Scores`, which cannot be instantiated, has two static methods. `static Couple<String, Integer> best(String[] names, int[] scores)`, where `scores[i]` is the number of points of `names[i]`, returns the leader: the team with the most points, with its points; on a tie, the team that comes first in the array wins. It throws `IllegalArgumentException` if the arrays are empty or have different lengths. `static Pair<Integer> minMax(int... values)` returns the smallest and the largest of the values, for example of the goal differences of the teams, which can be negative; it throws `IllegalArgumentException` when it receives no values.

```
Pair<String> match = Pair.of("Juventus", "Torino");     // home team, away team
Pair<Integer> result = new Pair<>(2, 1);                // home goals, away goals
String away = match.second();                           // no cast
int homeGoals = result.first();                         // auto-unboxing
IO.println(match + " " + away + " " + homeGoals + " " + match.swapped() + " "
        + match.contains("Juventus") + " " + result.contains(3));
result.set1st(3);                                       // the referee's report: 3-1
IO.println(result + " " + (result.first() + result.second()));
Couple<String, Integer> entry = new Couple<>("Juventus", 42);
Couple<Integer, String> byPoints = entry.swap();
Couple<String, Double> average = entry.withSecond(2.5);
IO.println(entry + " " + byPoints + " " + average + " "
        + entry.equals(new Couple<>("Juventus", 42)));
IO.println(Scores.best(new String[]{"Milan", "Inter", "Roma"}, new int[]{7, 9, 9}) + " "
        + Scores.minMax(5, -2, 8, 3));
```

Sample run:

```
(Juventus, Torino) Torino 2 (Torino, Juventus) true false
(3, 1) 4
<Juventus, 42> <42, Juventus> <Juventus, 2.5> true
<Inter, 9> (-2, 8)
```

6.2 ★★ Generic methods on arrays

Concepts: generic methods, bounded type parameters, `Comparable<? super T>`, comparators and predicates of a supertype, why `new T[]` does not compile and `Arrays.copyOf`, array covariance and `ArrayStoreException`.

A stationery shop keeps its data in arrays of different types: the tags of its catalogue (`String[]`), the pieces sold each day (`Integer[]`) and its products (`Product[]`). The programs of the shop need the same few operations on all of them: find an element, remove one, take the largest, add one at the end, count or select the elements with a property. Instead of writing each operation once per type, write it once as a generic method in the utility class `ArrayUtils`, which cannot be instantiated. A type parameter stands only for a class, which is why the counts are an `Integer[]` and not an `int[]`. Write the products as `record Product(String name, double price) implements Comparable<Product>`, whose natural order is by price. All the classes are in the package `ex6_2`, and the class `Main` contains the demo below.

R1 – Searching and editing

The method `<T> int indexOf(T[] a, T x)` returns the position of the first element equal to `x`, or `-1` if there is none; both `x` and the elements may be `null`. The method `<T> T remove(T[] a, T el)` is the slides' generic `remove`: it replaces the first element equal to `el` with `null`, and returns the removed element, or `null` if there was none. Thanks to the type parameter, the caller gets back a `String` from a `String[]` with no cast. The method `<T> void swap(T[] a, int i, int j)` exchanges two elements, for example two days whose counts were entered in the wrong cells.

R2 – The largest element

The method `<T extends Comparable<? super T>> T max(T[] a)` returns the largest element in the natural order of `T`: the last tag in alphabetical order, the highest count, the most expensive product. It skips the `null`s, which `remove` leaves behind, and returns `null` for an array that is empty or holds only `null`s. The bound makes `compareTo` available on `T`; written with `? super T`, it also accepts a class that inherits `compareTo` from a superclass.

Hint: the slides declare their `max` as `void`, although it returns a value: here it returns `T`.

The method `<T> T max(T[] a, Comparator<? super T> order)` does the same with the order given by a comparator, for example the products by name. Thanks to `? super T`, a comparator of a supertype works too, and the result keeps the type of the array: the maximum of an `Integer[]` found with a `Comparator<Object>` is still an `Integer`.

R3 – New arrays

The method `<T> T[] append(T[] a, T x)` returns a new array, one element longer, with `x` at the end: the count of a new day. The method `<T> int count(T[] a, Predicate<? super T> cond)` counts the elements that satisfy a condition, such as the busy days. The method `<T> T[] filter(T[] a, Predicate<? super T> cond)` returns a new array with the elements that satisfy the condition, in their order; its type must be that of the original array, so filtering a `Product[]` gives a `Product[]`, not an `Object[]`.

Hint: `new T[n]` does not compile. `Arrays.copyOf(a, n)` creates a new array of length `n` with the same run-time type as `a`, copying the elements that fit.

The last lines of the demo show why the shop's old code, which passed its `String[]` to methods that take an `Object[]`, failed only at run time: arrays are covariant, so a `String[]` can be used as an `Object[]`, and storing an `Integer` into it compiles, but throws `ArrayStoreException`.

```
String[] tags = {"office", "school", "art", "school"};
Integer[] sold = {13, 8, 21, 5};                        // pieces sold, Monday to Thursday
IO.println(ArrayUtils.indexOf(tags, "school") + " " + ArrayUtils.indexOf(sold, 99) + " "
        + ArrayUtils.max(tags) + " " + ArrayUtils.max(sold));
String removed = ArrayUtils.remove(tags, "school");   // no cast
IO.println(removed + " " + Arrays.toString(tags));
ArrayUtils.swap(sold, 0, 3);                            // Monday and Thursday were swapped
IO.println(Arrays.toString(ArrayUtils.append(sold, 34)) + " "
        + ArrayUtils.count(sold, n -> n > 10));
Product[] ps = {
    new Product("pen", 1.5), new Product("bag", 25.0), new Product("ink", 4.0)
};
IO.println(ArrayUtils.max(ps) + " "
        + ArrayUtils.max(ps, Comparator.comparing(Product::name)));
Product[] cheap = ArrayUtils.filter(ps, p -> p.price() < 5);
IO.println(Arrays.toString(cheap) + " " + cheap.getClass().getSimpleName());
Comparator<Object> byText = Comparator.comparing(Object::toString);
Integer m = ArrayUtils.max(sold, byText);               // T = Integer: see the notes
IO.println(m);
Object[] objects = tags;                                // arrays are covariant...
try {
    objects[0] = 42;                                    // ...so this compiles
} catch (ArrayStoreException e) {
    IO.println("ArrayStoreException: " + e.getMessage());
}
```

Sample run:

```
1 -1 school 21
school [office, null, art, school]
[5, 8, 21, 13, 34] 2
Product[name=bag, price=25.0] Product[name=pen, price=1.5]
[Product[name=pen, price=1.5], Product[name=ink, price=4.0]] Product[]
8
ArrayStoreException: java.lang.Integer
```

6.3 ★★ Your own generic list

Concepts: a generic container on an `Object[]`, unchecked casts and `@SuppressWarnings`, growing and shifting arrays, `remove(int)` versus `remove(Object)`, generic varargs and `@SafeVarargs`, a generic `Iterable` with an inner-class iterator, sorting with a `Comparator<? super E>`.

A medical practice keeps its waiting list in a program. Patients are added at the end as they arrive; an urgent one is inserted right after the patient who is about to be called in; the first one is called in; someone who gives up and leaves is taken off the list; at the end of the day the secretary prints the list in alphabetical order. Nobody knows in advance how many patients will come, so an array of fixed size is not enough, and the collections of `java.util` are not in the slides yet. Write your own growable list, the generic class `SimpleList<E>`, in the package `ex6_3`; the class `Main` contains the demo below.

The methods of `SimpleList` have the same names and meaning as those of `java.util.List`, so that switching to the library's `ArrayList`, when the collections arrive, costs nothing; only `equals` and `hashCode` stay those of `Object`. The exercises that follow reuse the class.

R1 – A list that grows

The elements live in an array of `Object`, because an `E[]` cannot be created. The array starts with 4 cells

and doubles its length whenever it is full and a new element arrives. For the demo, a package-private method `int capacity()` returns the current length of the array.

The method `boolean add(E e)` appends an element at the end and always returns `true`, as in `List`. The method `void add(int index, E e)` inserts an element at a position, shifting the following ones one place to the right: this is how an urgent patient goes second. An index equal to the size is allowed and appends. The method `E get(int index)` returns the element at a position, `E set(int index, E e)` replaces it and returns the old one, `int size()` returns the number of elements and `boolean isEmpty()` tells whether there are none. A position outside the list throws `IndexOutOfBoundsException` with the message `index <index>, size <size>`, e.g. `index 5, size 3`, where `<size>` is the current number of elements (in `add(int, E)` too).

Hint: reading an element back from the `Object[]` needs a cast to `E`, which the compiler cannot check and reports as *unchecked*. Keep that cast in one small private method, and silence the warning there with `@SuppressWarnings("unchecked")`. `Arrays.copyOf` grows the array, and `System.arraycopy` shifts its elements.

R2 – Searching and removing

The method `int indexOf(Object o)` returns the position of the first element equal to `o`, or `-1`; `o` and the elements may be `null`. The method `boolean contains(Object o)` tells whether the list holds such an element. Two methods remove an element, as in `List`: `E remove(int index)` removes the element at a position and returns it (the first patient is called in), while `boolean remove(Object o)` removes the first element equal to `o` and tells whether it found one (a patient leaves). The following elements shift one place to the left. Finally, `void clear()` empties the list.

On a `SimpleList<Integer>` the two `remove` methods are easy to confuse, exactly as with `List`: `remove(1)` removes the element at position 1, not the value 1.

R3 – Going through the list, sorting it, printing it

The class is declared `SimpleList<E> implements Iterable<E>`, so that a for-each loop goes through its elements. Its method `Iterator<E> iterator()` returns an object of a private inner class, which reads the elements of its list; at the end, the iterator's `next()` throws `NoSuchElementException`.

The method `void sort(Comparator<? super E> order)` sorts the list. Sort the occupied part of the array with `Arrays.sort(…, 0, size, order)`, as `ArrayList` does, so that the sort is stable (equal elements keep their order), and accept `null` to mean the natural order. The call needs a cast: see the theory notes. `toString()` returns the elements in square brackets, e.g. `"[Ann, Bob, Eva]"`, and `"[]"` for an empty list.

The static factory method `@SafeVarargs static <E> SimpleList<E> of(E... items)` builds a list from its arguments, such as the minutes waited by the last three patients.

Hint: a varargs parameter of a generic type makes the compiler warn about *heap pollution*, because it must create an array whose type it cannot check. `of` only reads the array, so the warning does not apply, and `@SafeVarargs` says so.

```
SimpleList<String> names = new SimpleList<>();
names.add("Zoe");
names.add("Ann");
names.add("Bob");
names.add(1, "Eva");
IO.println(names + " size=" + names.size() + " " + names.get(1) + " "
        + names.indexOf("Bob") + " " + names.contains("Max"));
for (int i = 0; i < 3; i++) {
    names.add("X" + i);
}
IO.println(names.size() + " " + names.capacity());
IO.println(names.remove(0) + " " + names.remove("X1") + " " + names.remove("nobody")
        + " " + names);
names.sort(Comparator.naturalOrder());
IO.println(names);
String initials = "";
for (String n : names) {
    initials += n.charAt(0);
}
IO.println(initials);
SimpleList<Integer> nums = SimpleList.of(5, 3, 8);
nums.set(0, nums.get(0) * 10);
int total = 0;
for (int n : nums) {
    total += n;
}
IO.println(nums + " total=" + total);
try {
    nums.get(3);
} catch (IndexOutOfBoundsException e) {
    IO.println("error: " + e.getMessage());
}
```

Sample run:

```
[Zoe, Eva, Ann, Bob] size=4 Eva 3 false
7 8
Zoe true false [Eva, Ann, Bob, X0, X2]
[Ann, Bob, Eva, X0, X2]
ABEXX
[50, 3, 8] total=61
error: index 3, size 3
```

6.4 ★★ Your own sorted registry

Concepts: a bounded type parameter on a class (`K extends Comparable<? super K>`), a static nested generic class, arrays of a generic type, binary search, the meaning of `Map.put`, counting with `getOrDefault`, why exceptions cannot be generic.

The stationery shop of 6.2 counts its stock per product name, and the OOP course keeps the texts of its past exams by year. Both need a table from keys to values, kept sorted by key: at the exam it is a `Map` (a `TreeMap`), which is not in the slides yet. Write your own, the generic class `Registry<K extends Comparable<? super K>, V>`; the later exercises use it where the exam solutions use a `Map`. All the classes are in the package `ex6_4`, which uses `SimpleList` (6.3), and the class `Main` contains the demo below. Errors are reported with two checked exceptions, `DuplicateKeyException` and `MissingKeyException`.

R1 – Keys and entries

The shop maps product names to the pieces in stock, a `Registry<String, Integer>`, and the archive

maps years to the subject of each exam, a `Registry<Integer, String>`. The values can be of any type, but the registry keeps its entries sorted by calling `compareTo` on the keys, which compiles only if every `K` has that method: this is what the bound on `K` says. It is `Comparable<? super K>` and not `Comparable<K>`, so that a key may also inherit `compareTo` from a supertype, as `LocalDate` does (it is a `Comparable<ChronoLocalDate>`). A `null` key cannot be compared, so, as in a `TreeMap`, every method that receives a key throws a `NullPointerException` for it.

Each key with its value is an object of a private static nested generic class `Entry<K, V>`. It is `static` because an entry needs no link to its registry: an inner class would give every entry a hidden reference to it. Being static, it belongs to no registry object, so it cannot use the registry's `K` and `V`: it declares type parameters of its own, which happen to have the same names. The entries are kept in an array sorted by key, with room for 4 at first and doubling when it is full, as in `SimpleList`. A key is found with binary search, and a new entry is inserted by shifting the following entries one place to the right; a removal shifts them back to the left.

Hint: `new Entry<K, V>[4]` does not compile (generic array creation): create a `new Entry<?, ?>[4]` and cast it to `Entry<K, V>[]`, suppressing the unchecked warning on that declaration only. A search that returns `-(insertion point) - 1` for a missing key, as `Arrays.binarySearch` does, tells both whether a key is there and where it would go.

R2 – Updating and reading

The methods have the names and the meaning of those of `Map`. The method `V put(K key, V value)` adds a key with its value, or replaces the value of a key already present, and returns the old value, or `null` for a new key, as `Map.put` does (so the result is `null` also when the old value was `null`). In the demo the shop records 10 pens and 4 ink bottles, then corrects the pens to 12. The method `V get(K key)` returns the value of a key, or `null` if the key is absent, and `V getOrDefault(K key, V otherwise)` returns `otherwise` instead, but only for a missing key, not for a key whose value is `null`. With it, counting takes one line: a delivery arrives as a string of scanned pieces, one product name per piece, and each piece adds one to the count of its product, starting from 0.

When a key must be new, the shop calls `void add(K key, V value) throws DuplicateKeyException`, which adds it, or leaves the table unchanged and throws `DuplicateKeyException` with the message `duplicate key <key>` if the key is present. When a key must be there, `V require(K key) throws MissingKeyException` returns its value, or throws `MissingKeyException` with the message `missing key <key>`. Finally, `boolean containsKey(K key)` tells whether a key is present, `V remove(K key)` removes a key and returns its value, or `null` if the key was absent, and `int size()` returns the number of entries.

The two exceptions are normal checked exceptions, subclasses of `Exception` with a constructor that takes the message. They cannot be generic (for example, a `DuplicateKeyException<K>` that holds the key): `catch` chooses by the run-time class of the exception, where type arguments no longer exist, so a generic class may not extend `Throwable`. The key reaches the caller in the message.

R3 – Keys, values and printing

The method `SimpleList<K> keys()` returns the keys and `SimpleList<V> values()` the values, both in key order. `keys()` plays the role of `Map.keySet()`, but both return new lists: changing them does not change the registry. `toString()` returns the entries in key order, as `{bag=3, cap=1}`, and `{}` for an empty registry. The demo's archive receives the 2017 exam first, and still lists the exams by year.

```
// main declared with: throws DuplicateKeyException, MissingKeyException
Registry<String, Integer> stock = new Registry<>();
IO.println(stock.put("pen", 10) + " " + stock.put("ink", 4) + " " + stock.put("pen", 12));
for (String w : "bag pen bag cap bag".split(" ")) {
    stock.put(w, stock.getOrDefault(w, 0) + 1);
}
IO.println(stock + " size=" + stock.size());
IO.println(stock.keys() + " " + stock.values() + " " + stock.get("zip") + " "
        + stock.containsKey("cap"));
try {
    stock.add("ink", 1);
} catch (DuplicateKeyException e) {
    IO.println("error: " + e.getMessage());
}
try {
    stock.require("zip");
} catch (MissingKeyException e) {
    IO.println("error: " + e.getMessage());
}
Registry<Integer, String> byYear = new Registry<>();
byYear.put(2017, "Delivery");
byYear.put(2014, "Trail");
byYear.put(2016, "Restaurants");
IO.println(byYear + " " + byYear.remove(2014) + " " + byYear.keys());
```

Sample run:

```
null null 10
{bag=3, cap=1, ink=4, pen=13} size=4
[bag, cap, ink, pen] [3, 1, 4, 13] null true
error: duplicate key ink
error: missing key zip
{2014=Trail, 2016=Restaurants, 2017=Delivery} Trail [2016, 2017]
```

6.5 ★★★ Generic functional helpers

Concepts: the generic functional interfaces of `java.util.function` (`Function`, `UnaryOperator`, `BinaryOperator`, `Predicate`, `Consumer`, `BiConsumer`, `Supplier`, `ToDoubleFunction`), wildcards in functional parameters, composing behaviour with `and`, `or`, `negate`, `andThen` and `compose`, a functional object with state, grouping by key.

Besides stationery (category `office`), the shop of 6.2 and 6.4 sells some food and travel goods, and it records each sale as a `record Sale(String product, String category, int qty, double price)`, where `price` is the price of one piece. Every week the manager asks a new question: which office products sold in quantity, the amount of each sale, the sales per category, how many are of cheap products. Instead of a new loop for each question, write the utility class `Fun`, which cannot be instantiated: its generic static methods contain the loop once and receive the question as a function, the way you will soon work with the library's streams. They use `SimpleList` (6.3) and `Registry` (6.4). All the classes are in the package `ex6_5`, and the class `Main` contains the demo below.

R1 – Selecting and transforming

The method `static <T, R> SimpleList<R> map(SimpleList<T> list, Function<? super T, ? extends R> f)` returns a new list with the result of `f` on each element, in order: with `Sale::product`, the products sold. The method `static <T> SimpleList<T> filter(SimpleList<T> list, Predicate<? super T> cond)` returns a new list with the elements that satisfy `cond`, in their order. Neither method changes its list. The method `static <T> void forEach(SimpleList<T> list, Consumer<? super T> action)` performs `action` on each element, in order.

The wildcards let the methods accept as many functions as possible: a function that accepts a supertype of `T` works on `T`s too, and one that returns a subtype of `R` still gives an `R`. So inputs are `? super T`, outputs `? extends R`, and a `Function<Object, String>` can map a `SimpleList<Sale>`.

The questions are built from simple functions with the default methods of the library's functional interfaces. With the predicates `office` and `big` (at least 5 pieces), `office.and(big)` selects the office products sold in quantity, and `office.negate().or(big)` the sales that are not of office products or are big. `f.andThen(g)` applies `f` and then `g` to its result, while `f.compose(g)` applies `g` first: the demo computes the amount of each sale and then writes it with two decimals, and takes the product of each sale and then the length of its name, for the shop's labels.

R2 – Totals

The method `static <T> T reduce(SimpleList<T> list, T identity, BinaryOperator<T> op)` combines the elements with `op` from left to right, starting from `identity`, which is also the result for an empty list: with `0` and `Integer::sum`, the total number of pieces sold. Its operator both receives and returns `T`s, so no wildcard fits. The method `static <T> double sum(SimpleList<T> list, ToDoubleFunction<? super T> value)` adds up a number computed from each element, such as the takings of the week. A `ToDoubleFunction` returns a plain `double` from `applyAsDouble`, so, unlike a `Function` to `Double`, it creates no object for each element.

R3 – Grouping

The exam's reports, such as "for each category, the list of…", are built exactly like `groupBy`. The method `static <T, K extends Comparable<? super K>> Registry<K, SimpleList<T>> groupBy(SimpleList<T> list, Function<? super T, ? extends K> key)` maps each key computed by `key` to the list of the elements with that key, in their original order: with `Sale::category`, the sales of each category. The key type must be comparable because a `Registry` keeps its keys sorted. The method `static <T, K extends Comparable<? super K>> Registry<K, Integer> countBy(SimpleList<T> list, Function<? super T, ? extends K> key)` counts the elements of each key instead: with a key that is `cheap` for a price below 2 and `normal` otherwise, how many sales are of cheap products. In both registries only the keys that occur are present.

Hint: to group, `get` the list of the element's key; the first time there is none, so create an empty one and `put` it; then add the element. To count, use the `getOrDefault` idiom of 6.4.

R4 – Generating

The method `static <T> SimpleList<T> generate(int n, Supplier<? extends T> next)` returns the results of `n` calls of `next.get()`, in the order of the calls (an empty list if `n` is 0 or less). The shop uses it with its generator of sale codes, a `Supplier` that must remember the last number it gave. A lambda has no fields, so the demo writes it as an anonymous class, which keeps a counter like the code generators of the exams; the same object goes on counting after `generate`. The demo also uses a `UnaryOperator`, a function from a type to the same type, to write the products sold in quantity in capitals for the shop window, and a `BiConsumer`, which receives two values, to print each category with its number of sales.

```
SimpleList<Sale> sales = SimpleList.of(
        new Sale("pen", "office", 10, 1.5),
        new Sale("ink", "office", 2, 4.0),
        new Sale("apple", "food", 6, 0.5),
        new Sale("bread", "food", 1, 2.25),
        new Sale("bag", "travel", 1, 25.0));
Predicate<Sale> office = x -> x.category().equals("office");
Predicate<Sale> big = x -> x.qty() >= 5;
IO.println(Fun.map(Fun.filter(sales, office.and(big)), Sale::product) + " "
        + Fun.map(Fun.filter(sales, office.negate().or(big)), Sale::product));
Function<Sale, Double> amount = x -> x.qty() * x.price();
Function<Sale, String> label = amount.andThen(a -> String.format(Locale.US, "%.2f", a));
```

```
IO.println(Fun.map(sales, label));
Function<String, Integer> length = String::length;
IO.println(Fun.map(sales, length.compose(Sale::product)));
IO.println(Fun.reduce(Fun.map(sales, Sale::qty), 0, Integer::sum) + " "
        + Fun.sum(sales, x -> x.qty() * x.price()));
Registry<String, SimpleList<Sale>> byCategory = Fun.groupBy(sales, Sale::category);
BiConsumer<String, Integer> show = (k, n) -> IO.print(k + ":" + n + " ");
for (String k : byCategory.keys()) {
    show.accept(k, byCategory.get(k).size());
}
IO.println();
IO.println(Fun.countBy(sales, x -> x.price() < 2 ? "cheap" : "normal"));
UnaryOperator<String> shout = w -> w.toUpperCase() + "!";
Fun.forEach(Fun.map(Fun.filter(sales, big), Sale::product),
        p -> IO.print(shout.apply(p) + " "));
IO.println();
Supplier<String> codes = new Supplier<>() {     // an object with its own state
    private int last = 0;

    @Override
    public String get() {
        return "S" + (++last);
    }
};
IO.println(Fun.generate(3, codes) + " " + codes.get());
```

Sample run:

```
[pen] [pen, apple, bread, bag]
[15.00, 8.00, 3.00, 2.25, 25.00]
[3, 3, 5, 5, 3]
20 53.25
food:2 office:2 travel:1
{cheap=2, normal=3}
PEN! APPLE!
[S1, S2, S3] S4
```

6.6 ★★★ Comparator factories, by hand and from the library

Concepts: comparator factories (generic methods that return a lambda), how `Comparator.comparing`, `comparingInt`, `reversed` and `thenComparing` work, the overflow of a subtracting comparator, comparators of a supertype, top-n on a copy.

The student office ranks students for scholarships and prints lists of them: by name, by average, the best `n`. A student is a `record Student(int id, String last, String first, double average)`. Each list needs a comparator, and the slides show how the library's `Comparator.comparing` and its friends are written: they are comparator factories, generic methods that receive a key or other comparators and return a new comparator, as a lambda. Write them yourself, with the right wildcards, in the utility class `Comparators`, which cannot be instantiated; the demo compares them with the library's. All the classes are in the package `ex6_6`, which uses `SimpleList` (6.3), and the class `Main` contains the demo below.

R1 – Comparing by a key

The method `static <T, U extends Comparable<? super U>> Comparator<T> comparing(Function<? super T, ? extends U> key)` returns the comparator that extracts the key of the two objects and compares the keys with `compareTo`: with `Student::last`, the order by last name. This is the library's signature: for any `T`, and any key type `U` that can be compared, it takes a function from `T` (or a supertype) to `U` (or a subtype), and gives back a comparator of `T`.

The method `static <T> Comparator<T> comparingInt(ToIntFunction<? super T> key)` does the same with an `int` key, such as the id, and compares the keys with `Integer.compare`. The slides subtract the keys, and so does the office's old comparator by id, `(a, b) -> a.id() - b.id()`, but the subtraction overflows. The demo gives two students ids near the limits of `int`: their difference wraps around and changes sign, so the old comparator puts Ugo before Bruno and Bruno before Eva, but Ugo after Eva. Sorted by id with both comparators, the students are in the right order only with yours.

R2 – Reversing and chaining

Scholarships go to the best averages, so the ranking starts from the highest. The method `static <T> Comparator<T> reverse(Comparator<T> order)` returns the opposite of `order` by swapping its two arguments; negating the result would fail on a comparator that returns `Integer.MIN_VALUE`. The library offers it as the default method `reversed()`. The method `static <T> Comparator<T> then(Comparator<T> first, Comparator<? super T> second)` returns the order `first`, whose ties are broken by `second`: `second` is called only when `first` returns 0. It is the library's `thenComparing`, which the office uses to sort by last name and then by first name, and to break the ties of the averages.

R3 – The best n

The method `static <T> SimpleList<T> top(SimpleList<T> list, int n, Comparator<? super T> order)` returns, as a new list, the first `n` elements in that order, or all of them if there are fewer: the exam's "best customers", here the students who get a scholarship. If `n` is negative it throws `IllegalArgumentException`. A report must not change the data it reads, so the original list must not change: sort a copy with the `sort` of `SimpleList`, which is stable, so that elements equal for `order` keep their original order. Since `order` is a `Comparator<? super T>`, a `Comparator<Object>` that compares objects by their text can rank a list of `Integer`s; by text, `100` comes before `9`.

R4 – The demo's helpers

In the demo class, next to `main`, write two helpers. The method `static <T> SimpleList<T> sorted(SimpleList<T> list, Comparator<? super T> order)` returns a sorted copy of the list, which stays unchanged. It too sorts with `SimpleList.sort`: with the old comparator the order depends on the sorting algorithm, and the sample run is the one given by `Arrays.sort`, which `SimpleList` uses. The method `static String names(SimpleList<Student> list)` returns the first names, in the order of the list, separated by spaces, as `Bruno Ugo Alice`. The last line of the demo shows that `top` has left the original list as it was.

```
SimpleList<Student> st = SimpleList.of(
        new Student(11, "Rossi", "Carla", 27.5),
        new Student(7, "Rossi", "Alice", 27.5),
        new Student(3, "Bianchi", "Bruno", 29.0),
        new Student(-2_000_000_000, "Neri", "Ugo", 18.0),
        new Student(2_000_000_000, "Verdi", "Eva", 30.0));
Comparator<Student> mine = Comparators.then(Comparators.comparing(Student::last),
        Comparators.comparing(Student::first));
Comparator<Student> lib = Comparator.comparing(Student::last)
        .thenComparing(Student::first);
IO.println(names(sorted(st, mine)) + " | " + names(sorted(st, lib)));
IO.println(names(sorted(st, Comparators.comparingInt(Student::id))) + " | "
        + names(sorted(st, (a, b) -> a.id() - b.id())));
IO.println(names(Comparators.top(st, 2,
        Comparators.reverse(Comparator.comparingDouble(Student::average)))));
IO.println(names(Comparators.top(st, 3, Comparator.comparingDouble(Student::average)
        .reversed().thenComparing(Student::last).thenComparing(Student::first))));
Comparator<Object> byText = Comparator.comparing(Object::toString);
// a Comparator<Object> is a Comparator<? super Integer>
IO.println(Comparators.top(SimpleList.of(10, 9, 100), 3, byText));
IO.println(names(st));
```

Sample run:

```
Bruno Ugo Alice Carla Eva | Bruno Ugo Alice Carla Eva
Ugo Bruno Alice Carla Eva | Eva Ugo Bruno Alice Carla
Eva Bruno
Eva Bruno Alice
[10, 100, 9]
Carla Alice Bruno Ugo Eva
```

6.7 ★★★ Wildcards and erasure

Concepts: invariance of generic types, `? extends`, `? super` and the rule "producer extends, consumer super", the unbounded `?`, type parameter or wildcard, a type parameter with two bounds (`&`), type erasure and its consequences, array generators.

A lab collects readings from two sensors: one gives whole numbers, kept in a `SimpleList<Integer>`, the other decimals, kept in a `SimpleList<Double>`. Both are then merged into a common log, a `SimpleList<Number>`, where the technicians also add readings by hand. The sum must work on all three lists; the other statistics compare readings, so they work on the sensors' lists, but not on the log, because a `Number` is not `Comparable`. Write them, with a few other tools, in the utility class `Lists`, which cannot be instantiated. All the classes are in the package `ex6_7`, which uses `SimpleList` (6.3) and `Pair` (6.1), and the class `Main` contains the demo below.

R1 – Statistics: `? extends`

A `SimpleList<Integer>` is not a `SimpleList<Number>`, although an `Integer` is a `Number`: if it were, a method could put a `Double` into a list of `Integer`s through the `SimpleList<Number>` reference. Generic types are *invariant*, so a `SimpleList<Number>` parameter would accept the log but neither sensor's list. Wildcards give back the flexibility, safely.

The method `static double sum(SimpleList<? extends Number> list)` returns the sum of the readings as a `double`. Its parameter is a list of some subtype of `Number`: the method only reads from it, and what it reads is a `Number`, whose `doubleValue()` it adds up. The method `static <T extends Comparable<? super T>> T max(SimpleList<? extends T> list)` returns the largest element, found with `compareTo`, and throws `NoSuchElementException` for an empty list, as `Collections.max`. Here `T` links the list to the result, so it must be a type parameter, while a type that appears only once,

as in `sum`, needs only a wildcard. The method `static <T extends Comparable<? super T>> boolean isSorted(SimpleList<T> list)` tells whether the elements are in non-decreasing order: equal neighbours are fine, and an empty list is sorted.

The spread of the readings needs more. The method `static <T extends Number & Comparable<? super T>> double range(SimpleList<T> list)` returns the largest element minus the smallest, as a `double`. Both are found with `compareTo`, which `T` has as a `Comparable`, and subtracted through `doubleValue`, which `T` has as a `Number`: so `T` has two bounds, joined by `&`, the class first. `Integer` and `Double` satisfy both. For an empty list it throws `NoSuchElementException`.

R2 – Filling the log: `? super`

The method `static <T> void copy(SimpleList<? super T> dst, SimpleList<? extends T> src)` appends the elements of `src` to `dst`, in order: this is how the readings of each sensor go into the log. The source produces `T`s, so it is `? extends T`; the destination consumes them, so it is `? super T`, a list of `T`s or of a supertype such as `Number`. This is the rule "producer `extends`, consumer `super`". `copy(a, a)` doubles `a`: read the size of `src` before adding, or the loop chases its own end.

The method `@SafeVarargs static <T> void addAll(SimpleList<? super T> dst, T... items)` appends the items, in order: the readings typed by hand, such as `7` and `8.5`. The annotation is correct because the method only reads the array of the items.

The method `static String describe(SimpleList<?> list)` returns the size and the content of a list, as `3 elements: [a, b, c]`, with the word `elements` whatever the size. It uses only `size()` and `toString()`, so the unbounded `SimpleList<?>`, a list of something, is enough: it describes the log and also a list of `Object`s.

R3 – Pairs and arrays

Each sensor has two alarm thresholds, kept as a `Pair<Integer>` (low, high). When a technician types them in the wrong order, the method `static <T> void sortPair(Pair<T> p, Comparator<? super T> order)`, the slides' pair sorting, puts them right: it swaps the two elements of the pair if the first comes after the second in `order`. Thanks to `? super T`, a `Comparator<Number>` that compares by value sorts a `Pair<Integer>`.

The lab exports its data to an old library that accepts only arrays. The method `static <T> T[] toArray(SimpleList<T> list, IntFunction<T[]> maker)` returns the elements of a list, in order, in a new array of the right type, such as a `String[]`. Since `new T[n]` does not compile, the caller passes a way to create the array, a function from a length to a new array, as in `toArray(list, String[]::new)`; the method calls `maker.apply` with the size of the list.

The last line of the demo shows type erasure: at run time there is a single class `SimpleList`, so a list of strings and one of integers have the same class, and `instanceof` can check `SimpleList<?>` but not a type argument.

```
SimpleList<Integer> ints = SimpleList.of(3, 1, 2);
SimpleList<Double> doubles = SimpleList.of(0.5, 0.25);
IO.println(Lists.sum(ints) + " " + Lists.sum(doubles));
IO.println(Lists.range(ints) + " " + Lists.range(doubles));
SimpleList<Number> numbers = new SimpleList<>();
Lists.copy(numbers, ints);                    // Integer -> Number
Lists.copy(numbers, doubles);                 // Double  -> Number
Lists.addAll(numbers, 7, 8.5);
IO.println(Lists.describe(numbers) + " sum=" + Lists.sum(numbers));
SimpleList<Object> objects = new SimpleList<>();
Lists.addAll(objects, "x", 1, 'c');
IO.println(Lists.describe(objects));
IO.println(Lists.isSorted(ints) + " " + Lists.isSorted(SimpleList.of("a", "b", "b")) + " "
        + Lists.max(ints));
Pair<Integer> p = new Pair<>(4, 2);
Comparator<Number> byValue = (a, b) -> Double.compare(a.doubleValue(), b.doubleValue());
Lists.sortPair(p, byValue);   // a Comparator<Number> sorts a Pair<Integer>
IO.println(p);
String[] arr = Lists.toArray(SimpleList.of("b", "a"), String[]::new);
IO.println(arr.length + " " + arr[0] + " " + arr.getClass().getSimpleName());
SimpleList<String> strings = new SimpleList<>();
IO.println((strings instanceof SimpleList<?>) + " "
        + (strings.getClass() == ints.getClass()));
```

Sample run:

```
6.0 0.75
2.0 0.25
7 elements: [3, 1, 2, 0.5, 0.25, 7, 8.5] sum=22.25
3 elements: [x, 1, c]
false true 3
(2, 4)
2 b String[]
true true
```

6.8 ★★★ Mock exam: a library

Concepts: the whole of Part 6 in a complete exam: `SimpleList` and `Registry` in place of `List` and `Map`, comparators with tie rules, grouping and counting, an enum, a checked exception, exam-style tests.

This is a complete exam in the style of the past ones, to do in about two hours.

A town library lends its books to its registered members. Write a program that keeps the catalogue of the books, the members and the loans, and produces the reports that the librarian needs. All the classes are in the package `ex6_8`; the facade is the class `Library`, and the class `Main` contains the demo below. All errors are reported with the checked exception `LibraryException`, and only the methods written below with `throws LibraryException` declare it; the messages are free. The collections are not in the slides yet, so use `SimpleList` (6.3) and `Registry` (6.4) wherever the exam solutions use `List` and `Map`. The solutions also contain the tests `TestR1_Books` … `TestR4_Reports`, written like the exam's: as at the exam, run them after each requirement.

R1 – Books

The library may own several copies of the same book. The method `String addBook(String title, String author, int year, int copies) throws LibraryException` adds a book to the catalogue, with its title, its author, its year of publication and the number of copies, and returns its code: `B1` for the first book, `B2` for the second, and so on. It throws an exception if the catalogue already has a book with the same title and the same author, compared exactly (the same title with another author is a different book), or if `copies` is less than 1. A refused book is not added and does not use up a code.

The method `String getBook(String code) throws LibraryException` returns the description of a book, in the format `<code> <title> - <author> (<year>)`, e.g. `B1 Dune - Herbert (1965)`. It throws an exception if the code is unknown.

The method `SimpleList<String> findByAuthor(String author)` returns the codes of the books of an author, whose name is compared ignoring case. The codes are sorted by year of publication and, for books of the same year, by title. The list is empty if the author has no books.

R2 – Members

The method `int addMember(String name)` registers a member and returns its id: 1 for the first member, 2 for the second, and so on. Two members may have the same name, and they get different ids. The method `String getMember(int id) throws LibraryException` returns the member in the format `<id> <name>`, e.g. `1 Mario Rossi`, and throws an exception if the id is unknown.

R3 – Loans

The method `int lend(String code, int memberId) throws LibraryException` lends a copy of a book to a member and returns the id of the loan: 1 for the first loan, 2 for the second, and so on. It throws an exception if the book or the member does not exist, if no copy of the book is available because they are all on loan, or if the member already has 3 active loans. A refused loan changes nothing and does not use up an id.

The method `void giveBack(int loanId) throws LibraryException` records that the copy of a loan has been brought back: the copy is available again, and the loan no longer counts among the active loans of its member. It throws an exception if the loan does not exist or has already been returned; in both cases nothing changes.

The method `int available(String code) throws LibraryException` returns how many copies of a book are not on loan, and throws an exception if the code is unknown. The method `LoanStatus status(int loanId) throws LibraryException` returns the state of a loan, a value of `enum LoanStatus { ACTIVE, RETURNED }`: a loan is `ACTIVE` when it is made and becomes `RETURNED` when it is given back. It throws an exception if the loan is unknown.

R4 – Reports

The method `SimpleList<String> mostLent(int n)` returns the `n` books that have been lent most often, counting the returned loans too, each in the format `<code>:<count>`, e.g. `B2:3`. They are sorted by decreasing count and, for the same count, by the number in the code, so that `B2` comes before `B10`. Books never lent are left out; if fewer than `n` books have been lent, the list contains all of them.

The method `Registry<String, SimpleList<String>> activeLoans()` returns, for each name of a member with active loans, the titles of the books that the members with that name have on loan now, in alphabetical order. Members with the same name share one entry, and members without active loans do not appear.

The method `Registry<Integer, Integer> titlesPerDecade()` returns, for each decade (1960, 1970, ...), how many books of the catalogue were published in it. It counts the books, not their copies, and only the decades with at least one book appear.

```
// main declared with: throws LibraryException
Library lib = new Library();
String dune = lib.addBook("Dune", "Herbert", 1965, 2);
String found = lib.addBook("Foundation", "Asimov", 1951, 1);
String robots = lib.addBook("I, Robot", "Asimov", 1950, 1);
String neu = lib.addBook("Neuromancer", "Gibson", 1984, 1);
int mario = lib.addMember("Mario Rossi");
int anna = lib.addMember("Anna Bianchi");
IO.println(lib.getBook(dune) + " | " + lib.findByAuthor("asimov") + " | "
        + lib.getMember(anna));
int l1 = lib.lend(dune, mario);
int l2 = lib.lend(dune, anna);
int l3 = lib.lend(found, anna);
try {
    lib.lend(dune, mario);
} catch (LibraryException e) {
    IO.println("error: " + e.getMessage());
}
lib.giveBack(l1);
int l4 = lib.lend(dune, mario);
lib.lend(neu, mario);
IO.println(l1 + " " + l2 + " " + l3 + " " + l4 + " " + lib.status(l1) + " "
        + lib.status(l4) + " " + lib.available(dune));
IO.println(lib.mostLent(3));
IO.println(lib.activeLoans());
IO.println(lib.titlesPerDecade());
try {
    lib.giveBack(l1);
} catch (LibraryException e) {
    IO.println("error: " + e.getMessage());
}
```

Sample run (the messages are free; these are the ones in the solution):

```
B1 Dune - Herbert (1965) | [B3, B2] | 2 Anna Bianchi
error: no copy of B1 available
1 2 3 4 RETURNED ACTIVE 0
[B1:3, B2:1, B4:1]
{Anna Bianchi=[Dune, Foundation], Mario Rossi=[Dune, Neuromancer]}
{1950=2, 1960=1, 1980=1}
error: loan 1 already returned
```